PHP end-of-term exam (2020-21-1)

Due No Due Date **Points** None

Important information

- You have **120 minutes** to complete the exam. We provide you **additional 30 minutes** for filling in the `README.md` (see template below), reading the problems, downloading the materials, compressing and uploading your solution.
- Solutions must be uploaded to ELTE Canvas. **The system closes at 18:30 (Hungarian time) and there will be no opportunity to hand in your solution after the aforementioned timepoint**.
- You may use **any kind of material** (including documentation, lectures, class exercises, cheat sheets, etc) during the exam, **except human help** (synchronous, asynchronous, chat, forums, etc)!
- The problems are independent and can be solved **in any order**.
- To begin your work you should first **download the starting package we have prepared**. Each tasks has its own directory in the package. All tasks are somewhat prepared. Other than the provided files, you're free to use any other files (to store data, organize your code, implement AJAX/fetch), but make sure your solutions are in the correct folder!

```
task 1: 1/index.php
task 2: 2/index.php
task 3: 3/index.php
task 4: 4/index.php
```

- The starting package contains a template called `README.md` where you should fill in your name and Neptun code (the <> characters must be removed). **Submissions without a properly filled `README.md` file will not be graded!**

```
<Name>
<Neptun ID>
This solution was submitted and prepared by the student stated above for the assignment of the Web p
rogramming course.
I declare that this solution is my own work.
I have not copied or used third party solutions.
I have not passed my solution to my classmates, neither  made it public.
Students' regulation of Eötvös Loránd University (ELTE Regulations Vol. II. 74/C. § ) states
that as long as a student presents another student's work - or at least the significant part of it -
as his/her own performance, it will count as a disciplinary fault. The most serious consequence of a
disciplinary fault can be dismissal of the student from the University.
```

- Each directory contains a file named `TASKS.md`. Replace the space in `[ ]` with an `x` where you have (even partially) succeeded to complete the task. This will make us easier to find your solution when we grade your exam.

Task 1: Time table

The goblins devised a plan to steal treasure from the Kobold Kingdom. There's only one problem: they don't know when will they have time to do this. Create a page which displays the daily activities for each goblin! (The life of goblins is really boring, they do the same things in the same time each day.) **Before starting to code, look at and try to understand the data structure in `1/index.php`!**

- a. (2 points) Display the activities in a table where the first row is the name of each goblin! (keys from the `$goblins` associative array.)
- b. (2 points) The goblins count time from sunrise to sunset, divided into 8 hours. Sunrise is the start of hour 0, and the sun sets exactly at 7:59. In accordance with this, display the hours in the first column of the table from 0 to 7!
- c. (2,5 points) We know the following information for each activity: when it starts (`startHour`) and what is it's ID (`activityKey`). Each acticity is one hour long. The names and difficulties of each activity can be found in the `$activities` array. For a given cell, if the given goblin (first row) has an activity in the given hour (first column), display the name of the activity there!
- d. (1,5 points) The difficulty of each activity is measured as a real number between 0 and 1. Color every task:
  - easier than 0.5 (0.5 included) to `lightgreen`
  - between 0.5 and 0.8 to `orange`
  - harder than 0.8 (0.8 included) to `red` If a cell has no activity in it, leave it white.

Example output (different from your data)

Időbeosztás

ÓRA	LO'LIN	AR'IN	REN'GR	VOL'LI
0	játék		vadászat	programozás
1	játék		vadászat	programozás
2				
3	játék	alvás	zsákmányolás	
4	játék	alvás	zsákmányolás	alvás
5	játék	alvás	család	család
6	játék	alvás		alvás
7				

Task 2: Work organizing

The goblins want to plan how they will steal the treasure. They will need the following:

- worker goblins
- the name and the rank of the leader of worker goblins
- at least as much shovels as workers, but preferably twice as many.

Based on this, create a page (`2/index.php`) where the URL accepts parameters the following way:

- how many worker goblins are coming (a number)

- who is the leader of the workers: a string with at least one space, where the last word is always the rank of the goblin (e.g. `Snuch Nawdow grandchief` and `Vik'thor general` are both possible inputs)
- how many shovels are they bringing (a number)

A possible URL is `index.php?goblins=5&chief=Snuch Nawdow grandchief&shovels=7`

Output the following, based on the input:

- a. (2 points) If the `goblins` parameter doesn't exist, is not a number, not an integer or not positive: `Invalid goblin quantity!`
- b. (1,5 points) If the `chief` parameter doesn't exist or doesn't have at least one space: `Invalid leader!`
- c. (2 points) Check the leader's rank:
  - If it's not a goblin rank: `Invalid rank!`
  - If it's not `grandchief` at least: `Rank too low!`
  - Possible goblin ranks (from lowest to highest): lil'goblin, lil'chief, grandchief, general, tribechief
- d. (1,5 points) If the `shovels` parameter doesn't exist, is not a number, not an integer, or negative: `Invalid shovel quantity!`
  - If the number of shovels is less than the number of goblins: `Too few shovels!` (you only have to check this if the number was valid)
- e. (1 point) If all parameters are okay: `Ready to go!`, and if the number of shovels is at least twice as much as the number of goblins: `We got to the treasure quickly!`

Example input-output pairings

```
url: index.php?goblins=5&chief=Snuch Nawdow grandchief&shovels=7
output:
- Ready to go!

url: index.php?goblins=5&chief=Snuch Nawdow grandchief&shovels=10
output:
- We got to the treasure quickly!

url: index.php
output:
- Invalid goblin quantity!
- Invalid chief!
- Invalid shovel quantity!

url: index.php?goblins=notanumber&chief=nospace&shovels=notanumber
output:
- Invalid goblin quantity!
- Invalid chief!
- Invalid shovel quantity!

url: index.php?goblins=-5&chief=Snuch Nawdow grandchief&shovels=10
output:
- Invalid goblin quantity!

url: index.php?goblins=16.2&chief=Snuch Nawdow grandchief&shovels=10
```

```
output:
- Invalid goblin quantity!

url: index.php?goblins=16&chief=Snuch Nawdow grandchief&shovels=10
output:
- Too few shovels!

url: index.php?goblins=5&chief=Snuch Nawdow secretary&shovels=10
output:
- Invalid rank!

url: index.php?goblins=5&chief=Snuch Nawdow lil'chief&shovels=10
output:
- Rank too low!
```

*These input-output pairs are just examples, it is not enough if your solution works correctly for **only** these!*

Task 3: Treasure records

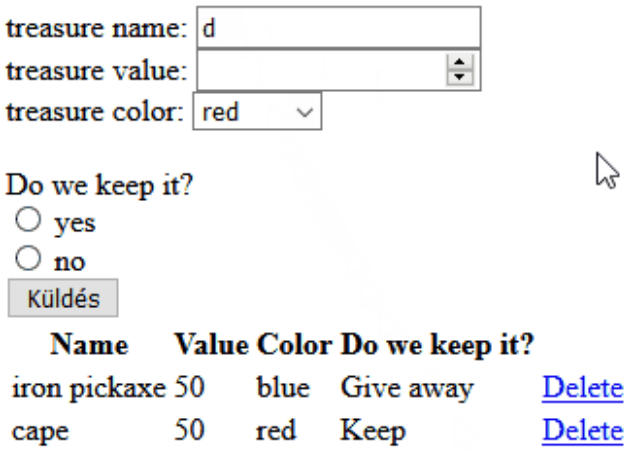
The goblins have acquired the treasure and want to categorise them with the help of a form, where you can input the following?

- Name of the treasure (text)
- Value of the treasure (number)
- Color of thre treasure (list of options: red, orange, yellow, green, blue, purple)
- The option to keep or give away the treasure (radio buttons)

You can assume that the input data is correct (it exists, is not empty), however, make sure you save the correct types! (numbers as numbers, boolean as boolean etc.)!

- a. (1 point) Arriving on the page, a form displays with all required fields.
- b. (1,5 points) When filled, the form data gets saved to a file (with type information!) After a succesful save, display the form again.
- c. (1,5 points) Under the form, display all previously saved treasure in a table. This table should be displayed every time the page is (re)loaded, even the first time (when it is still empty).
- d. (2 points) A saved treasure can be deleted with a `Delete` link displayed in it's respective row.
- e. (2 points) Implement the data storing in a way that checks if a treasure with this name has already been stored. If yes, instead of storing it's data again, update it!

How it works



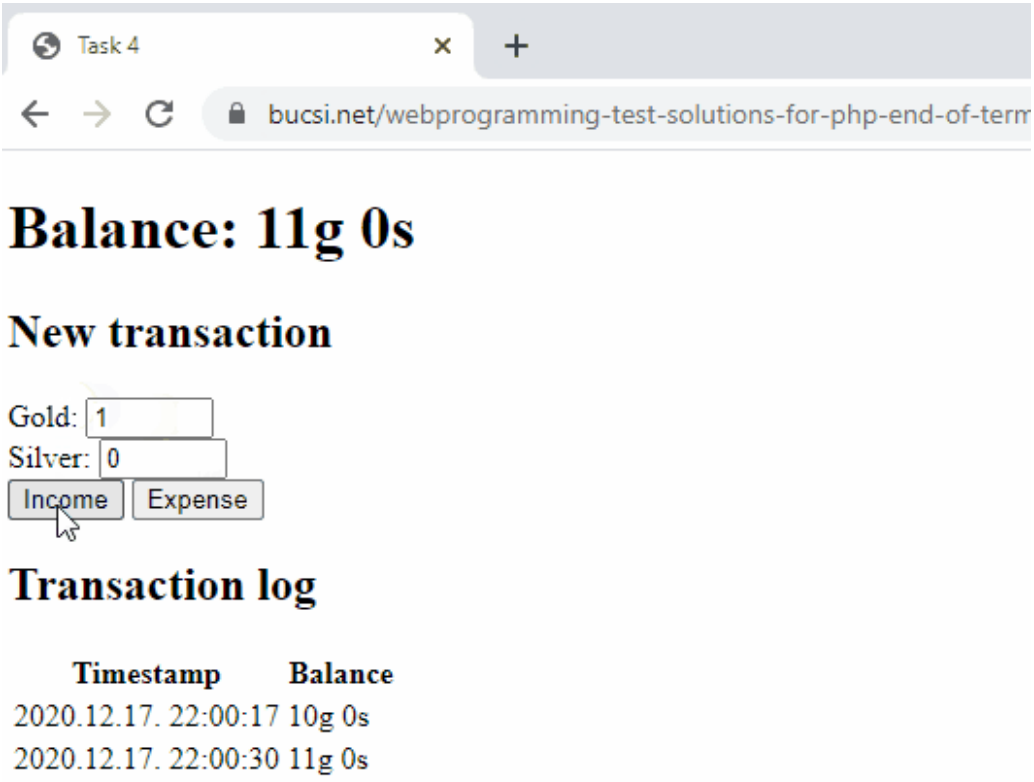
Task 4: Goblin treasury

Some goblins had one crucial task left: they held an auction and sold off some of the treasure. The profits from the auction were recorded by the goblin treasurers with the help of a bookkeeping application. Create this application!

Goblins count money in gold coins (abbreviated as g), and each gold coin can be divided up to 12 silver coins (abbreviated as s). Store the treasury's balance in the session! If the session does not exist, the starting balance is *10g 0s*. Transactions are logged in a table on the site.

- a. (1 point) When (re)loading the page, display a table with the header and one row: the date and time of loading and the starting balance. (Note: the starting balance is not always *10g 0s*, the session can contain data from previous page loads!)
- b. (1,5 points) Clicking the *income* button increases the balance **without reloading the page. All data processing should happen on the server-side. Use AJAX/fetch to facilitate data exchange between the server and the client!** (See subtask e. for displaying the modified balance!)
- c. (1 point) Always group silver coins to gold! This means that after each update, the new balance can only be *Xg [0..11]s*! For example: *1g 3s + 2g 10s = 4g 1s*
- d. (1,5 points) Clicking the *expense* button on the page subtracts from the balance. If the new balance would be less than 0, don't update the balance, treat it as the transaction didn't happen. (See subtask e. for displaying the modified balance!)
- e. (1,5 points) After each *income* or *expense* button press, generate a new table row with the current date, time and the new balance! **All data should come from the server, you can only do the rendering in JS!**
- f. (1,5 points) If the transaction did not change the balance, do not create a new row! (This happens when the expense was greater than the balance, or *0g 0s* was sent to the server.)

How it works (balance should not be display at the top of the page)



JavaScript mid-term exam			
Criteria	**Ratings**		**Pts**
1/b. Output the contents of the field to the console while typing using an input event.	**1.0 pts** **Teljes megoldás**	**0.0 pts** **Nem értékelhető**	1.0 pts
1/c. Display the number of characters in the editor to the HTML element with the ID number-of-characters.	**1.0 pts** **Teljes megoldás**	**0.0 pts** **Nem értékelhető**	1.0 pts
1/d. Display the number of rows in the editor to the HTML element with ID number-of-	**1.0 pts** **Teljes**	**0.0 pts** **Nem**	1.0 pts

rows. (Rows are separated by \n)	**megoldás**	**értékelhető**	
1/e. Output the number of vowels in the first row to the console while typing. The Hungarian vowels are: aáeéiíoóöőuúüű.	**1.0 pts** **Teljes megoldás**	**0.0 pts** **Nem értékelhető**	1.0 pts
1/f. List the number of vowels per row in the HTML list with ID vowels-per-row.	**1.5 pts** **Teljes megoldás**	**0.0 pts** **Nem értékelhető**	1.5 pts
1/g. If the number of rows is 3 and they consist of 5-7-5 syllables then add the style class haiku to the p element that contains the input field. This will make the background green and a button should appear. Remove the class if the haiku conditions aren't met.	**1.5 pts** **Teljes megoldás**	**0.0 pts** **Nem értékelhető**	1.5 pts
1/h. If you apply the style correctly, a button appears. When the button is clicked you need to add the content of the editor to the <pre> elements inside the element with the ID haikus.	**1.0 pts** **Teljes megoldás**	**0.0 pts** **Nem értékelhető**	1.0 pts
2/b. Using JavaScript display the matrix stored in variable matrix to the HTML element table-container as a table (using table, tr, td elements) when the "Generate table" button is clicked.	**1.5 pts** **Teljes megoldás**	**0.0 pts** **Nem értékelhető**	1.5 pts
2/c. When a cell is clicked, output its row,column coordinates to the console.	**1.0 pts** **Teljes megoldás**	**0.0 pts** **Nem értékelhető**	1.0 pts
2/d. Solve task c. by registering only one event listener (delegating).	**1.5 pts** **Teljes megoldás**	**0.0 pts** **Nem értékelhető**	1.5 pts
2/e. The following tasks must be recalculated and redisplayed after every cell click. It might be helpful and timesaving the use array functions as it takes only one row to solve each task with them. If you couldn't solve tasks b. or c. then you may use a hardcoded matrix to continue.	**1.0 pts** **Teljes megoldás**	**0.0 pts** **Nem értékelhető**	1.0 pts
2/f.			

Display (using words "Yes" or "No") whether any bird has been observed in the first settlement (whether the first row of the matrix contains any value other than zero) to the element with ID task-1.	**1.0 pts** **Teljes megoldás**	**0.0 pts** **Nem értékelhető**	1.0 pts
2/g. Display the number of settlements where more than 10 birds of the same species have been observed (how many rows contain at least one value greater than 10) to the element with ID task-2.	**1.0 pts** **Teljes megoldás**	**0.0 pts** **Nem értékelhető**	1.0 pts
2/h. Display the row index (starting from 1 (ONE)) of a settlement where no bird has been observed at all (the whole row only contains zeros) to the element with ID task-3. Display the text "No" if no such settlement exists.	**1.0 pts** **Teljes megoldás**	**0.0 pts** **Nem értékelhető**	1.0 pts
3/b. Create the function random(a, b) that returns a random integer from the closed interval [a, b]. Output a random integer between -10 and 10 (inclusive) to the console.	**1.0 pts** **Teljes megoldás**	**0.0 pts** **Nem értékelhető**	1.0 pts
3/c. Fill an array of length 20 with random values between -5 and 5 (inclusive) and output the filled array to the console.	**1.5 pts** **Teljes megoldás**	**0.0 pts** **Nem értékelhető**	1.5 pts
3/d. Draw the elements of the array to the canvas following these rules. Draw a broken line from the vectical center of the left edge to the center of the right edge so that the first line segment ends 10px to the right and at the height which corresponds to the first element of the array. The next line segment ends after an additional 10px to the right and above or below based on the second element.	**1.5 pts** **Teljes megoldás**	**0.0 pts** **Nem értékelhető**	1.5 pts
3/e. Your lines should be 3px wide and gray.	**1.0 pts** **Teljes megoldás**	**0.0 pts** **Nem értékelhető**	1.0 pts
3/f. When the button "Change" is clicked, add a random integer between -1 and 1 (inclusive) to every element of the array and redraw the new state to the canvas (after clearing it first).	**1.5 pts** **Teljes megoldás**	**0.0 pts** **Nem értékelhető**	1.5 pts
3/g. When the button "Start animation" is clicked, make the change automatic and	**1.5 pts** **Teljes**	**0.0 pts** **Nem**	

continuous. Keep changing the elements of the array and redrawing the state to the canvas using some kind of timer mechanism (requestAnimationFrame or setInterval).	**megoldás**	**értékelhető**	1.5 pts
4/b. When a step is clicked add the style class done to it. Use delegation if you can (see task h.).	**1.0 pts** **Teljes megoldás**	**0.0 pts** **Nem értékelhető**	1.0 pts
4/c. Do not apply the style class done to tasks when they are clicked.	**1.0 pts** **Teljes megoldás**	**0.0 pts** **Nem értékelhető**	1.0 pts
4/d. If a step is clicked all uncompleted steps before it also get added to style class done.	**1.5 pts** **Teljes megoldás**	**0.0 pts** **Nem értékelhető**	1.5 pts
4/e. If a list item that has already been marked as completed is clicked then remove the style class done only if none of the steps after the clicked one are completed. Help: you can use nextElementSibling method to iterate through the siblings.	**1.0 pts** **Teljes megoldás**	**0.0 pts** **Nem értékelhető**	1.0 pts
4/f. If all steps of a certain task are completed then add the style class done to the task itself.	**1.0 pts** **Teljes megoldás**	**0.0 pts** **Nem értékelhető**	1.0 pts
4/g. If there are uncompleted steps then the task cannot be marked as completed.	**1.0 pts** **Teljes megoldás**	**0.0 pts** **Nem értékelhető**	1.0 pts
4/h. Solve the above tasks by registering event listeners to one element only (delegating). Help: you can use the CSS selector parent > child to differentiate the tasks and steps.	**1.5 pts** **Teljes megoldás**	**0.0 pts** **Nem értékelhető**	1.5 pts
			Total Points: 32.0